

# PROMPT CURSOR — EventsCar / GP-DEV-SINISTRO

## (Sistema Completo)

**Como usar:** Copie todo este documento para uma nova conversa no Cursor (modo Agent) e peça:  
"Implemente/mantenha este sistema conforme a especificação abaixo."

**Produção:** <https://eventos.escsistemas.com>

**Repositório:** devgpsc/GP-DEV-SINISTRO

**Supabase Project ID:** yxawavenbognqiihaesh

### 1. Visão do produto

Sistema SaaS multi-tenant para **gestão de sinistros automotivos** (associados/proteção veicular), cobrindo o ciclo:

Sinistro → Cotação (RFQ) → Matriz comparativa → OC (Compra) → Entrega → Conclusão

**Personas:**

- Operador: registra sinistros, cotações, acompanha entregas
- Gerente: aprova OCs por escrito, relatórios, equipe
- Admin empresa: configurações, API, auditoria
- Super Admin plataforma: tenants, planos SaaS

### 2. Stack tecnológica

| Camada        | Tecnologia                                        |
|---------------|---------------------------------------------------|
| Frontend      | React 18 + TypeScript + Vite 5                    |
| Estilo        | Tailwind CSS 3                                    |
| Ícones        | lucide-react                                      |
| Roteamento    | react-router-dom v6                               |
| Backend dados | Supabase (PostgreSQL + Auth + Storage + Realtime) |
| API REST      | Vercel Serverless (/api/v1/*)                     |
| Auxiliar      | /api/cnpj, /api/vehicles, /api/llm                |
| Deploy        | GitHub Actions → Vercel + Supabase migrations     |
| Planilhas     | xlsx (export matriz)                              |

#### Variáveis de ambiente

**Frontend ( .env ):**

```
VITE_SUPABASE_URL=https://xxx.supabase.co
VITE_SUPABASE_ANON_KEY=eyJ...
```

```
VITE_API_BASE_URL=/api
VITE_APP_URL=https://eventos.escsistemas.com
```

#### Vercel (API serverless):

```
SUPABASE_URL=https://xxx.supabase.co
SUPABASE_SERVICE_ROLE_KEY=eyJ...
GEMINI_API_KEY=...
APIBRASIL_TOKEN=...
```

### 3. Arquitetura

```
flowchart TB
    subgraph Cliente
        Browser[React SPA]
    end
    subgraph Vercel
        SPA[Static dist]
        APIv1[/api/v1 REST]
        APIaux[/api/cnpj vehicles llm]
    end
    subgraph Supabase
        Auth[Auth PKCE]
        PG[(PostgreSQL RLS)]
        Storage[Storage buckets]
        RT[Realtime]
    end
    Browser --> SPA
    Browser --> Auth
    Browser --> PG
    Browser --> RT
    APIv1 --> PG
    Browser --> APIaux
```

#### Multi-tenant

- Toda tabela operacional tem `tenant_id`
- RLS: `is_platform_super_admin()` OR `tenant_id IN get_my_tenant_ids()`
- Trigger `set_default_tenant_id()` no INSERT
- Tenant ativo em `localStorage (sb-autoclaims-tenant-id)`

### 4. Estrutura de pastas

```
|— api/                # Serverless Vercel
| |— _lib/             # auth, supabase admin, http helpers
| |— v1/               # API REST pública (API Key)
| |— cnpj/             # Proxy BrasilAPI
| |— vehicles/         # Lookup placa
```

|                       |                                    |
|-----------------------|------------------------------------|
| └─ llm/               | # Proxy Gemini                     |
| components/           | # UI reutilizável                  |
| context/              | # AuthContext, ToastContext        |
| hooks/                | # useEventTypes, etc.              |
| pages/                | # Uma página por módulo            |
| services/             | # supabaseClient, *Service.ts      |
| supabase/migrations/  | # SQL versionado                   |
| utils/                | # impressão, defaults, matrixPrint |
| docs/                 | # Documentação técnica             |
| └─ .github/workflows/ | # CI/CD                            |

## 5. Módulos funcionais (pages/)

| Rota           | Arquivo          | Responsabilidade                                                |
|----------------|------------------|-----------------------------------------------------------------|
| /              | Dashboard.tsx    | KPIs, gráficos, atalhos                                         |
| /eventos       | Events.tsx       | Sinistros: vínculo associado/veículo, anexos, cota participação |
| /cotacoes      | Quotations.tsx   | RFQ wizard, matriz, cadastro rápido sinistro                    |
| /compras       | Purchases.tsx    | OCs agrupadas por caso, aprovação escrita, auditoria            |
| /entregas      | Deliveries.tsx   | Fila entregas, marcar entregue, editar/excluir                  |
| /fornecedores  | Suppliers.tsx    | Parceiros, CNPJ/CEP auto, avaliações                            |
| /associados    | Associates.tsx   | Clientes PF/PJ, CNPJ lookup                                     |
| /veiculos      | Vehicles.tsx     | Frota, lookup placa                                             |
| /catalogo      | Catalog.tsx      | Peças e serviços padronizados                                   |
| /relatorios    | Reports.tsx      | BI (requer view_reports)                                        |
| /configuracoes | Settings.tsx     | IA, equipe, API keys, auditoria                                 |
| /saas-admin    | SaaSAdmin.tsx    | Super admin tenants/planos                                      |
| /login         | Login.tsx        | Auth email/OAuth                                                |
| /register      | Register.tsx     | Nova empresa ou convite                                         |
| /auth/callback | AuthCallback.tsx | OAuth/email confirm                                             |

## 6. Fluxos de negócio críticos

### 6.1 Sinistro (Events)

1. Selecionar **Associado / Terceiro** e **Veículo** (ou cadastro rápido inline)
2. Tipo sinistro (configurável em Settings → Tipos, inclui **Acordo**)
3. **Cota de participação do veículo** (campo numérico opcional)
4. Anexos multimídia → Supabase Storage bucket `event-attachments`
5. Viewer in-browser ( `FileViewerModal` ) para imagem/vídeo/PDF

- 6. Exclusão via RPC `delete_event_cascade` (valida retorno)

6.2 Cotação (Quotations)

- 1. Wizard: selecionar sinistro OU cadastro rápido (nome, placa, CNPJ, cota, anexos)
- 2. Itens do catálogo (peças/serviços)
- 3. Convidar fornecedores
- 4. Matriz ( `MatrixTable` ): preços manuais/simulados, seleção manual (não auto)
- 5. Impressão matriz: preview **Paisagem** ou **Lista** ( `utils/matrixPrint.ts` )

6.3 Compra (Purchases)

- 1. `quotationService.processPurchase()` gera 1 OC por fornecedor (status `Gerada` )
- 2. Aprovação exige **justificativa por escrito** + permissão `approve_purchases`
- 3. Trigger DB valida permissão e nota obrigatória
- 4. **Histórico auditoria** ( `purchase_order_history` ) automático + UI timeline
- 5. Preview impressão OC antes de print ( `purchaseOrderPrint.ts` )

6.4 Entregas (Deliveries)

- 1. OCs aprovadas aparecem na fila
- 2. Marcar entregue: responsável + observação → status OC `Recebida`
- 3. Editar/excluir entregas (ActionModal estilizado)
- 4. Histórico movimentações em `deliveries.movement_history` (JSONB)

7. Autenticação e segurança

Login

- Supabase Auth PKCE + storage híbrido ( `authStorage.ts` )
- Google OAuth → `/auth/callback`
- Sem membership → logout forçado (sem "modo rápido")
- `PrivateRoute` : exige user + membership ou `super_admin`

Permissões ( `services/accessControl.ts` )

| Flag                           | Efeito                   |
|--------------------------------|--------------------------|
| <code>approve_purchases</code> | Aprovar/cancelar OC      |
| <code>delete_records</code>    | Excluir registros        |
| <code>financial_view</code>    | Valores financeiros      |
| <code>view_reports</code>      | <code>/relatorios</code> |
| <code>manage_users</code>      | Equipe                   |

Gerente = `Admin` / `Gerente` `profile` OU `owner` / `admin` `membership`.

Rotas guardadas

- `PermissionRoute` em `/configuracoes` , `/relatorios`
- `AdminRoute` em `/saas-admin`

RPCs sensíveis

- `complete_registration` , `accept_invite` , `get_tenant_members`
- `update_tenant_member_profile` , `detach_tenant_member`
- `create_tenant_api_key` , `revoke_tenant_api_key` , `validate_api_key`
- `delete_event_cascade` , `delete_tenant_cascade`

Documentação: `docs/SEGURANCA-E-PERMISSOES.md`

---

## 8. Banco de dados (principais tabelas)

| Tabela                                     | Propósito                          |
|--------------------------------------------|------------------------------------|
| <code>saas_tenants</code>                  | Empresas clientes                  |
| <code>organization_members</code>          | Usuário ↔ tenant                   |
| <code>profiles</code>                      | Perfil, role, permissions JSONB    |
| <code>invitations</code>                   | Convites pendentes                 |
| <code>events</code>                        | Sinistros                          |
| <code>event_history</code>                 | Histórico status sinistro          |
| <code>event_attachments</code>             | Anexos sinistro                    |
| <code>associates</code>                    | Associados/terceiros               |
| <code>vehicles</code>                      | Veículos                           |
| <code>suppliers</code>                     | Fornecedores                       |
| <code>catalog_items</code>                 | Catálogo peças/serviços            |
| <code>quotations</code>                    | Cotações RFQ                       |
| <code>quotation_items</code>               | Itens da cotação                   |
| <code>quotation_supplier_prices</code>     | Preços matriz                      |
| <code>quotation_purchase_selections</code> | Seleção manual compra              |
| <code>purchase_orders</code>               | Ordens de compra                   |
| <code>purchase_order_items</code>          | Itens OC (normalizado)             |
| <code>purchase_order_history</code>        | <b>Auditoria OC</b>                |
| <code>deliveries</code>                    | Entregas                           |
| <code>audit_logs</code>                    | Auditoria global                   |
| <code>api_keys</code>                      | Chaves integração REST             |
| <code>saas_settings</code>                 | Config tenant (IA, tipos sinistro) |

### Campos recentes:

- `events.participation_quota` , `quotations.participation_quota` , `quotations.attachments`

- `purchase_orders.approval_note` , `approved_by` , `approved_at`

## 9. API REST v1 (integração externa)

**Base URL:** `https://eventos.escsistemas.com/api/v1`

**Autenticação:** Header `Authorization: Bearer evsc_live_...` ou `X-API-Key`

**Gerar chave:** Configurações → API / Integrações

| Método | Endpoint                      | Auth | Descrição                                    |
|--------|-------------------------------|------|----------------------------------------------|
| GET    | <code>/health</code>          | Não  | Health check                                 |
| GET    | <code>/</code>                | Não  | Metadados endpoints                          |
| GET    | <code>/events</code>          | Sim  | Listar sinistros                             |
| GET    | <code>/quotations</code>      | Sim  | Listar cotações ( <code>?id=</code> detalhe) |
| GET    | <code>/purchase-orders</code> | Sim  | Listar OCs ( <code>?id=</code> com history)  |
| GET    | <code>/deliveries</code>      | Sim  | Listar entregas                              |
| GET    | <code>/associates</code>      | Sim  | Listar associados                            |

**Implementação:** `api/v1/*.js` + `api/_lib/auth.js` (valida hash via RPC `validate_api_key` )

Documentação completa: `docs/API-INTEGRACAO.md`

### APIs auxiliares (sem API Key)

| Endpoint                                     | Uso                         |
|----------------------------------------------|-----------------------------|
| GET <code>/api/cnpj/lookup?cnpj=</code>      | Razão social, CEP, endereço |
| GET <code>/api/vehicles/lookup?plate=</code> | Dados veículo               |
| POST <code>/api/llm/generate</code>          | Proxy IA (Gemini)           |

## 10. Serviços frontend ( services/ )

| Arquivo                              | Função                                 |
|--------------------------------------|----------------------------------------|
| <code>supabaseClient.ts</code>       | Cliente Supabase PKCE                  |
| <code>eventService.ts</code>         | CRUD sinistros, anexos, delete cascade |
| <code>quotationService.ts</code>     | Matriz, processPurchase, simulação     |
| <code>purchaseOrderService.ts</code> | Histórico auditoria OC                 |
| <code>attachmentService.ts</code>    | Upload/viewer anexos Storage           |
| <code>lookupService.ts</code>        | CNPJ, CEP, placa                       |

|                         |                                   |
|-------------------------|-----------------------------------|
| quickRegisterService.ts | Cadastro rápido associado/veículo |
| accessControl.ts        | Resolve permissões UI             |
| auditService.ts         | Logs audit_logs                   |
| apiKeyService.ts        | CRUD chaves API                   |

## 11. Componentes UI importantes

| Componente      | Uso                                                  |
|-----------------|------------------------------------------------------|
| PremiumModal    | Modais formulário (sinistro)                         |
| ActionModal     | Confirmações estilizadas (substituir window.confirm) |
| PermissionRoute | Guard rotas por permissão                            |
| PrivateRoute    | Guard autenticação                                   |
| MatrixTable     | Matriz cotação comparativa                           |
| FileViewerModal | Visualizar vídeo/imagem/PDF                          |
| Layout          | Sidebar navegação                                    |

**Padrão visual:** rounded-2xl/3xl, font-black uppercase tracking-widest labels, blue-600 primary, cards brancos border slate-200.

## 12. Realtime

Subscriptions Supabase em:

- Purchases.tsx → purchase\_orders , quotations
- Deliveries.tsx → deliveries , purchase\_orders

## 13. CI/CD

.github/workflows/deploy.yml :

1. **build** — npm ci && npm run build
2. **migrate** (main) — supabase db push
3. **deploy-production** — Vercel com retry + inspect Ready
4. **deploy-preview** — PRs

Autor git deploy: 257080210+devgpesc@users.noreply.github.com

## 14. Migrations essenciais (ordem lógica)

1. 20240401\_saas\_architecture\_v2.sql — multi-tenant
2. 20260525114135\_enforce\_operational\_tenant\_rls.sql — RLS tenant
3. 20260524232239\_harden\_tenant\_user\_permissions.sql — RPCs equipe

4. 20260716220000\_sinistro\_entregas\_anexos.sql — anexos, approval OC
  5. 20260716230000\_delete\_event\_cascade\_rpc.sql — delete sinistro
  6. 20260717010000\_participation\_quota\_quotation\_attachments.sql — cota RFQ
  7. 20260717030000\_po\_history\_api\_keys\_security.sql — auditoria OC, API keys, guards
- 

## 15. Regras de implementação (para o Agent Cursor)

1. **Responda em português** na UI e toasts
  2. **Nunca** usar `window.confirm` — usar `ActionModal`
  3. **Sempre** verificar `{ error }` em mutations Supabase
  4. **Respeitar** `access.*` antes de ações sensíveis
  5. **Manter** isolamento tenant — nunca query sem RLS
  6. **Cadastro rápido** reutilizar `quickRegisterService`
  7. **Anexos** reutilizar `attachmentService` + `ATTACHMENT_ACCEPT`
  8. **Impressão** abrir preview no navegador antes de `window.print()`
  9. **Commits** só quando usuário pedir; mensagens em português
  10. **Build** deve passar: `npm run build`
- 

## 16. Checklist funcional (aceite)

- ☐ Login/logout/convite/registro empresa
  - ☐ Sinistro com associado/terceiro, veículo, cota, anexos vídeo
  - ☐ Cotação RFQ com anexos e cadastro rápido
  - ☐ Matriz impressão paisagem/lista
  - ☐ OC aprovação por escrito + auditoria timeline
  - ☐ Entregas editar/excluir/marcar entregue
  - ☐ Fornecedor CNPJ+CEP automático
  - ☐ Catálogo CRUD com `ActionModal` delete
  - ☐ API REST v1 com chaves em Settings
  - ☐ Permissões bloqueiam rotas e ações
  - ☐ Deploy CI verde
- 

## 17. Prompt de execução (copiar e colar)

Você é o agente principal do EventsCar (GP-DEV-SINISTRO).

Stack: React + TypeScript + Vite + Supabase + Vercel serverless.  
Multi-tenant com RLS. Produção: eventos.escsistemas.com.

Implemente/mantenha TODOS os módulos descritos neste documento:

- Sinistros, Cotações, Compras (OC+auditoria), Entregas, Fornecedores, Associados, Veículos, Catálogo, Relatórios, Configurações, SaaS Admin.

Requisitos obrigatórios:

1. Segurança: `PermissionRoute`, `accessControl`, `triggers` DB em OCs
2. API REST v1 com API Keys (`events`, `quotations`, `purchase-orders`, `deliveries`, `associates`)
3. `ActionModal` em exclusões; sem `window.confirm`



4. Anexos multimídia com FileViewerModal
5. Impressão sempre com preview no browser
6. Migrations Supabase versionadas em supabase/migrations/
7. npm run build sem erros antes de commit

Ao alterar schema, crie migration SQL.

Ao criar endpoints, atualize docs/API-INTEGRACAO.md.

Documente decisões em docs/ quando relevante.

Comece lendo: App.tsx, AuthContext, accessControl.ts, supabase/migrations mais recentes.

---

## 18. Referências de arquivos-chave

pages/Events.tsx  
pages/Quotations.tsx  
pages/Purchases.tsx  
pages/Deliveries.tsx  
pages/Settings.tsx  
components/MatrixTable.tsx  
components/ActionModal.tsx  
services/quotationService.ts  
services/eventService.ts  
services/purchaseOrderService.ts  
utils/matrixPrint.ts  
utils/purchaseOrderPrint.ts  
api/v1/  
supabase/migrations/  
.github/workflows/deploy.yml  
docs/API-INTEGRACAO.md  
docs/SEGURANCA-E-PERMISSOES.md  
docs/PROMPT-ANEXOS-VIDEO-CURSOR.md

---

*Documento gerado para replicação e manutenção do EventsCar via Cursor IDE.*

*Versão: 2026-07-17*